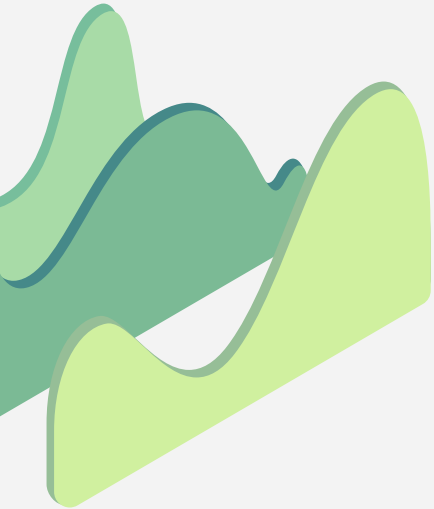
200Lab Education

SQL for Beginners

Nga Vu - Data Mentor







200Lab Education

Nga Vu - Data Mentor

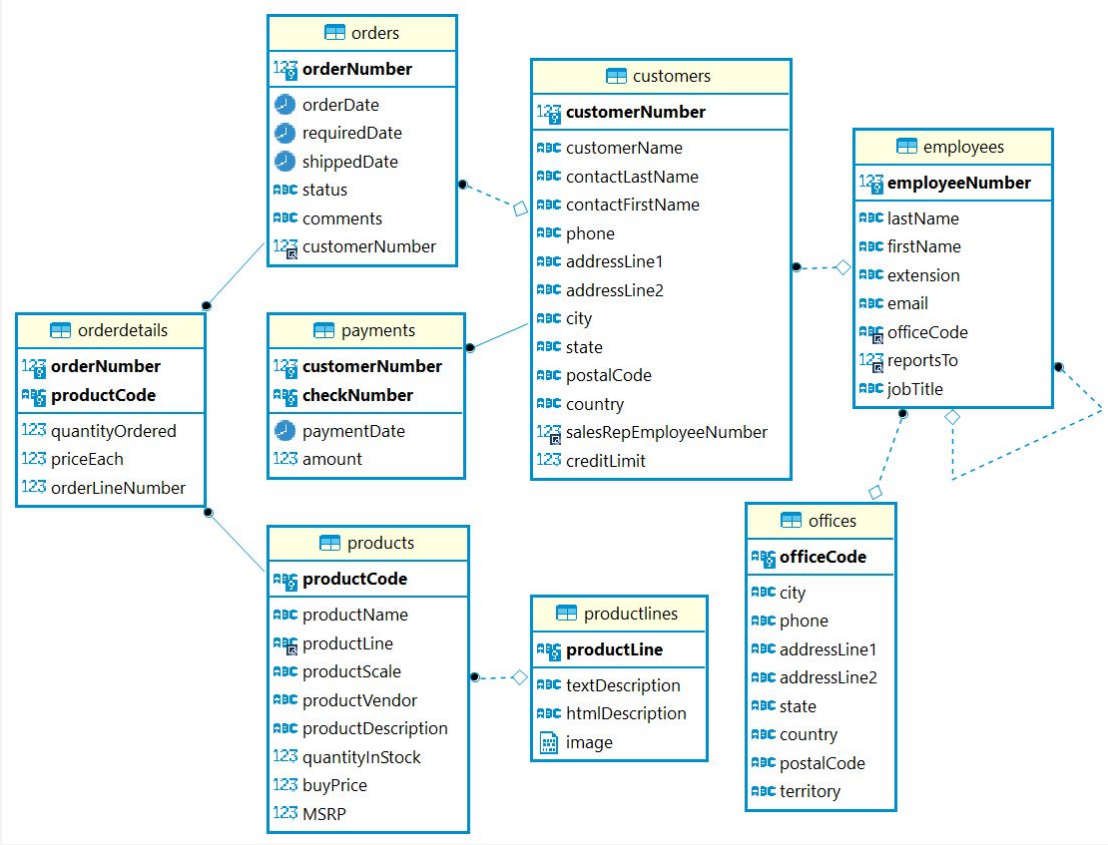
Query Multiple Tables

Join & Union

Primary Key & Foreign Key

- **A primary key** is used to ensure data in the specific column is **unique**.
- **A foreign key** is a column or group of columns in a relational database table that **provides a link between data in two tables**

	orderNumber	productCode
1	10,100	S18_1749
2	10,100	S18_2248
3	10,100	S18_4409
4	10,100	S24_3969
5	10,101	S18_2325

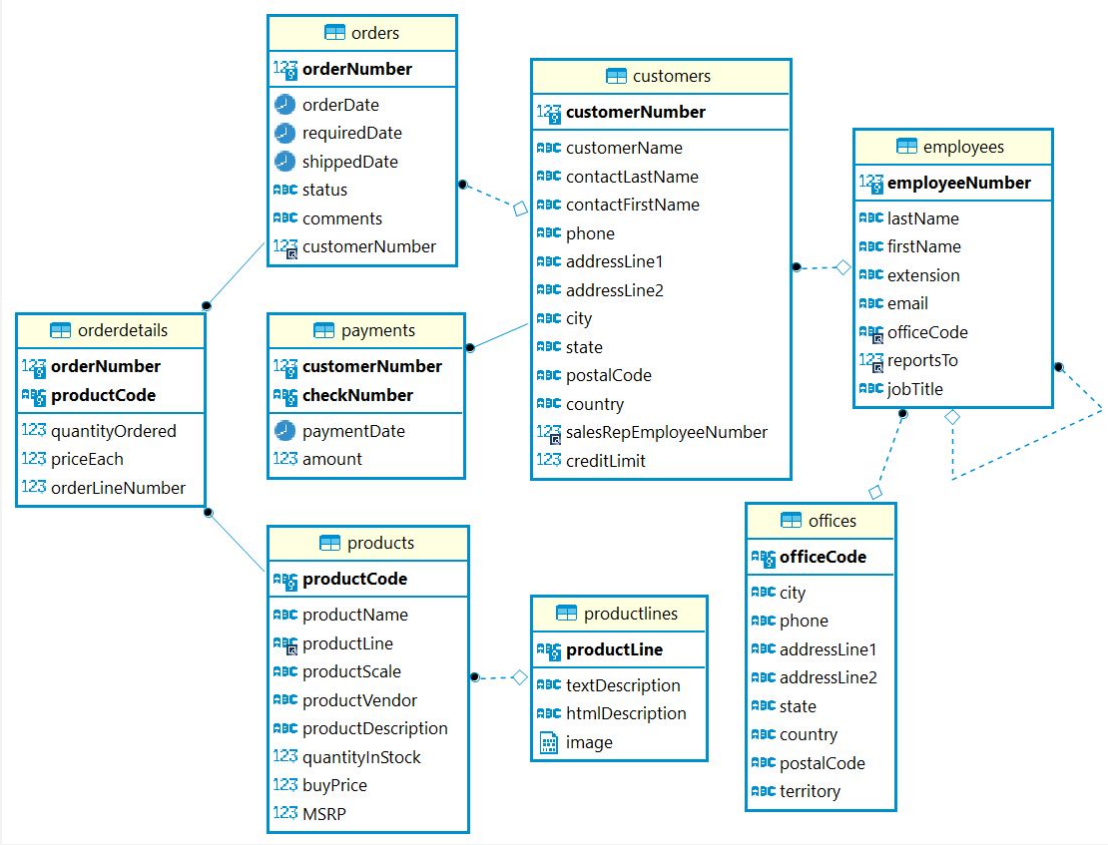


What is a Join ?

Queries against a single table are certainly not rare, but you will find that **most of your queries** will require two, three, or even more tables.

- Total sales , No. orders by customer's state ?
- Calculate employee's commission
- Calculate monthly sales

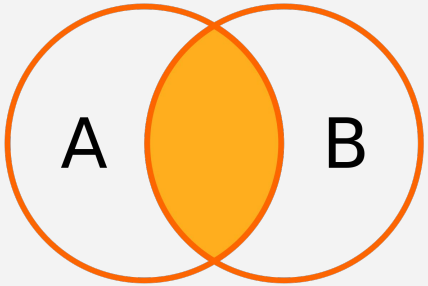
A JOIN is a means for **combining columns from multiple tables** by using values common to each

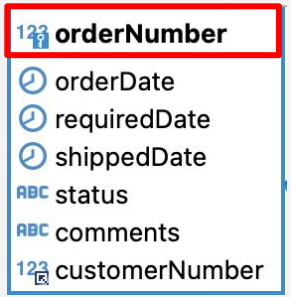


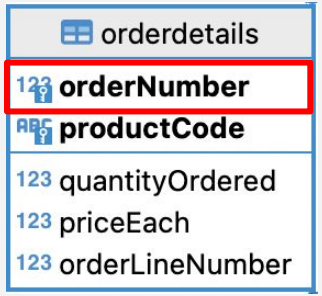
INNER JOIN - JOIN

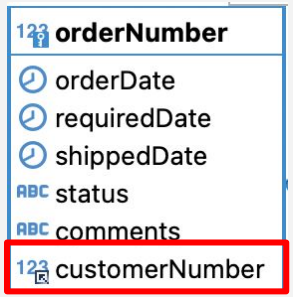
A **JOIN** clause is used to combine rows from two or more tables that have **matching values** (intersection) in two or more tables

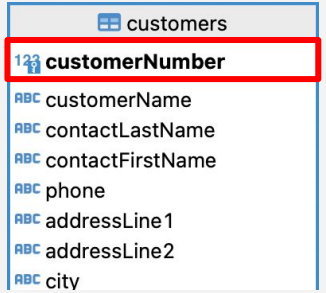
```
SELECT column_list
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name
```





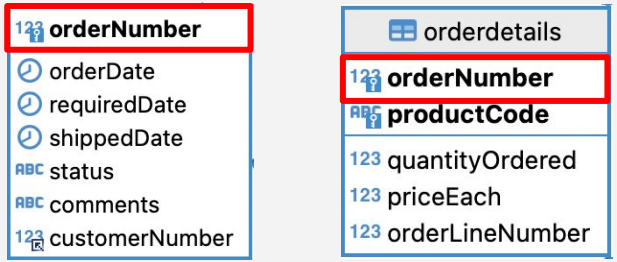






INNER JOIN

Calculate **Grand Total** of each order




```
SELECT o.orderNumber,
od.priceEach*od.quantityOrdered
'GrandTotal'
FROM orders o JOIN orderdetails od
        ON o.orderNumber =
od.orderNumber;
```

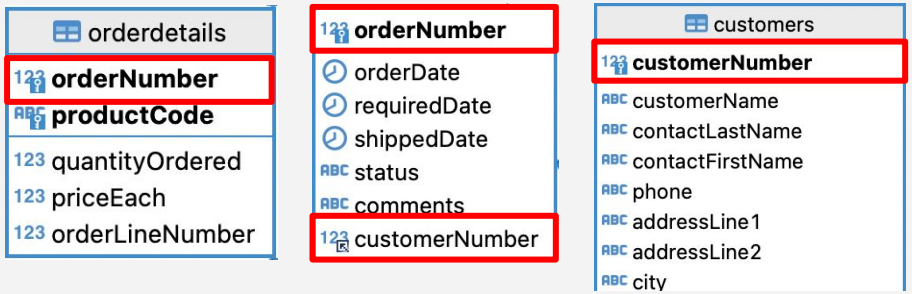
	orderNumber	GrandTotal
1	10,123	3,138.46
2	10,123	5,282.64
3	10,123	3,986.84
4	10,123	2,163.5
5	10,298	4,128.54
6	10,298	1,938.24
7	10,345	1,676.14
8	10,124	3,213

```
SELECT o.orderNumber,
SUM(od.priceEach*od.quantityOrdered) 'GrandTotal'
FROM orders o JOIN orderdetails od
        ON o.orderNumber = od.orderNumber
GROUP BY orderNumber;
```

	orderNumber	Ctrl+click to
1	10,100	10,223.83
2	10,101	10,549.01
3	10,102	5,494.78
4	10,103	50,218.95
5	10,104	40,206.2
6	10,105	53,959.21
7	10,106	52,151.81
8	10,107	22,292.62

INNER JOIN

Calculate **Total Revenue** of each customer, get top 5

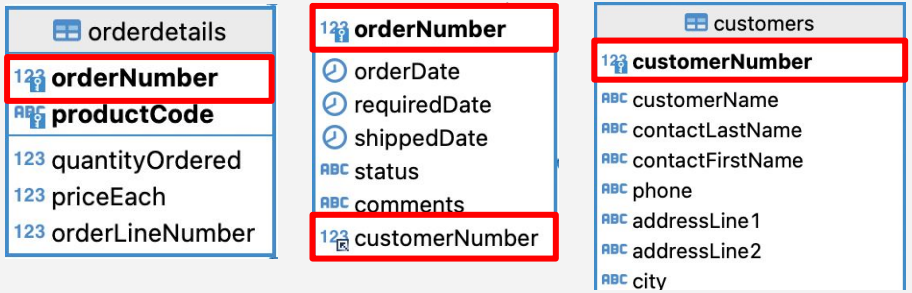


	customerName	Total Revenue
1	Euro+ Shopping Channel	22,680
2	Mini Gifts Distributors Ltd.	16,746.58
3	Australian Collectors, Co.	5,159.19
4	Muscle Machine Inc	4,800.74
5	Land of Toys Inc.	4,575.97

```
SELECT c.customerName,SUM(od.priceEach* od.quantityOrdered) 'Total Revenue'
FROM orders o JOIN orderdetails od
        ON o.orderNumber = od.orderNumber
        JOIN customers c ON c.customerNumber = o.customerNumber
GROUP BY c.customerName
ORDER BY 2 DESC LIMIT 5
```

INNER JOIN

Calculate **Total Revenue, No. orders** of each customer

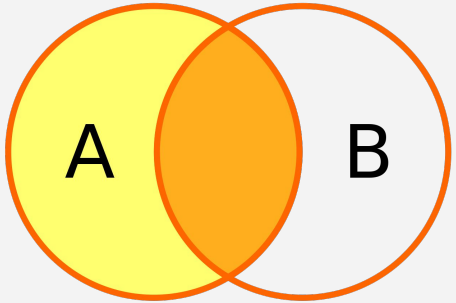


	customerName	No. orders	Total Revenue
1	Alpha Cognac	3	1,749.4
2	Amica Models & Co.	2	2,522.69
3	Anna's Decorations, Ltd	4	4,314.94
4	Atelier graphique	3	601.49
5	Australian Collectables, Ltd	3	1,816.84
6	Australian Collectors, Co.	5	5,159.19
7	Australian Gift Network, Co	3	1,528.57

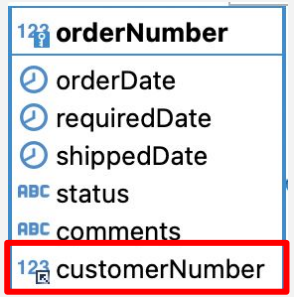
```
SELECT c.customerName,COUNT(DISTINCT o.orderNumber) 'No.
orders',SUM(od.priceEach) 'Total Revenue'
FROM orders o JOIN orderdetails od
        ON o.orderNumber = od.orderNumber
        JOIN customers c ON c.customerNumber = o.customerNumber
GROUP BY c.customerName
```

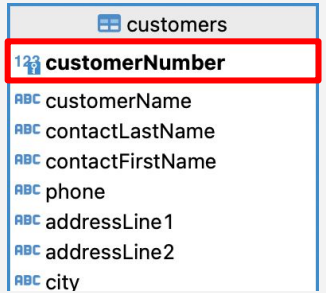
LEFT JOIN

The **LEFT JOIN** returns all rows from the left table, even if there are **no matches** in the right table




```
SELECT column_list
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name
```





LEFT JOIN

Show customer names and their No. orders
(include who didn't buy anything)

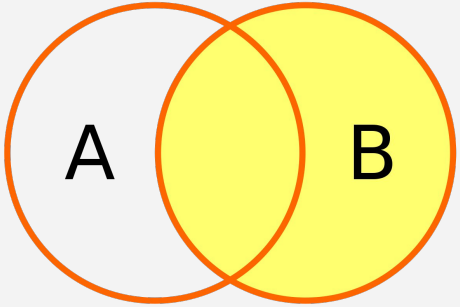
```
SELECT c.customerName, COUNT(DISTINCT o.orderNumber)
'No. orders'
FROM customers c LEFT JOIN orders o
        ON o.customerNumber  =  c.customerNumber
GROUP BY 1
```

	customerName	No. orders
1	Alpha Cognac	3
2	American Souvenirs Inc	0
3	Amica Models & Co.	2
4	ANG Resellers	0
5	Anna's Decorations, Ltd	4
6	Anton Designs, Ltd.	0

RIGHT JOIN

The **RIGHT JOIN** returns all rows from the right table, even if there are **no matches** in the left table

```
SELECT column_list
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name
```





RIGHT JOIN

Show customer names and their No. orders
(include who didn't buy anything)

```
SELECT c.customerName, COUNT(DISTINCT o.orderNumber)
'No. orders'
FROM orders o RIGHT JOIN customers c
        ON o.customerNumber  =  c.customerNumber
GROUP BY 1
```

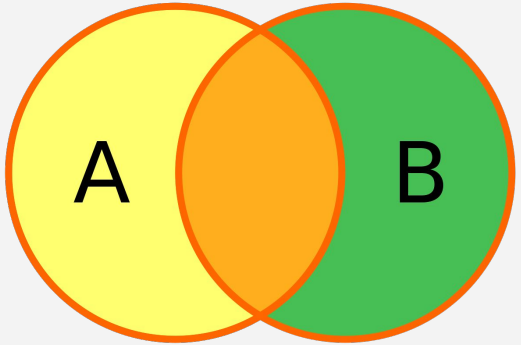
	customerName	No. orders
1	Alpha Cognac	3
2	American Souvenirs Inc	0
3	Amica Models & Co.	2
4	ANG Resellers	0
5	Anna's Decorations, Ltd	4
6	Anton Designs, Ltd.	0

FULL OUTER JOIN (**)

The **FULL OUTER JOIN** keyword returns all records when there is a match in left (A) or right (B) table records.

MySQL does not support Full Outer Join

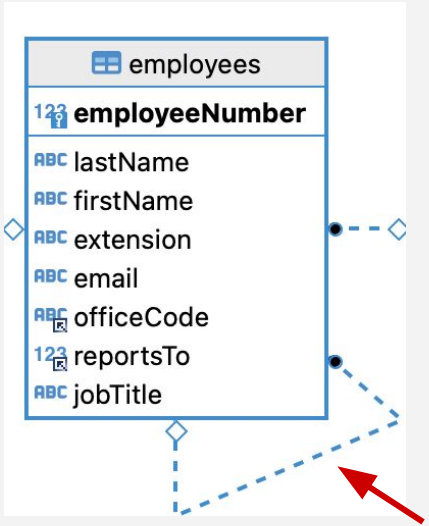
(**) : Out of scope of this course



SELF JOIN

A self join is a regular join, but the table is **joined with itself**

```
SELECT column_list
FROM A  table1
JOIN A table2
ON table1.column_name = table2.column_name
```



SELF JOIN

Show employee names and their managers

```
SELECT e1.employeeNumber,
       CONCAT(e1.firstName,' ', e1.lastName) Name,
       CONCAT(e2.firstName,' ', e2.lastName) managerName
FROM employees e1
       JOIN employees e2
       ON e1.reportsTo = e2.employeeNumber
```

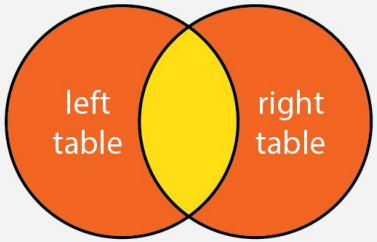
nber	Name	mangerName
1,611	Andy Fixter	William Patterson
1,612	Peter Marsh	William Patterson
1,619	Tom King	William Patterson
1,621	Mami Nishi	Mary Patterson
1,625	Yoshimi Kato	Mami Nishi
1,702	Martin Gerard	Gerard Bondur

```
SELECT e1.employeeNumber,
       CONCAT(e1.firstName,' ', e1.lastName) Name,
       CONCAT(e2.firstName,' ', e2.lastName) managerName
FROM employees e1
       LEFT JOIN employees e2
       ON e1.reportsTo = e2.employeeNumber;
```

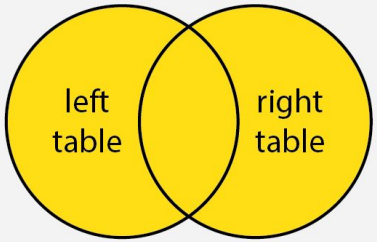
ber	Name	mangerName
1,002	Diane Murphy	[NULL]
1,056	Mary Patterson	Diane Murphy
1,076	Jeff Firrelli	Diane Murphy
1,088	William Patterson	Mary Patterson
1,102	Gerard Bondur	Mary Patterson
1,143	Anthony Bow	Mary Patterson
1,165	Leslie Jennings	Anthony Bow

JOIN Key Takeaway

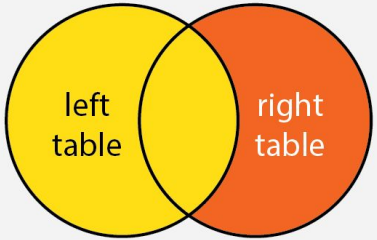
INNER JOIN



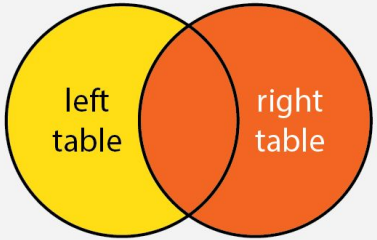
FULL JOIN



LEFT JOIN



RIGHT JOIN



JOIN Tips & Tricks

When JOIN, Avoid duplicates

```
SELECT *
FROM address a JOIN customer c
ON c.customer_id = a.customer_id
```



```
SELECT *
FROM shipping s JOIN orders o ON o.order_id = s.order_id
        JOIN customer c ON o.customer_id = c.customer_id
        JOIN address a ON s.address_id = a.address_id
```

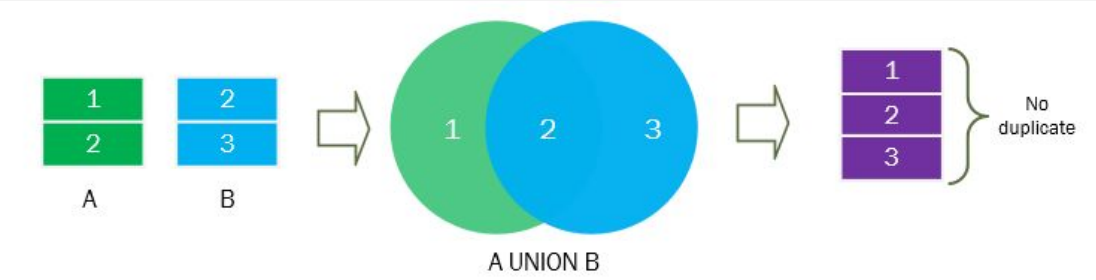


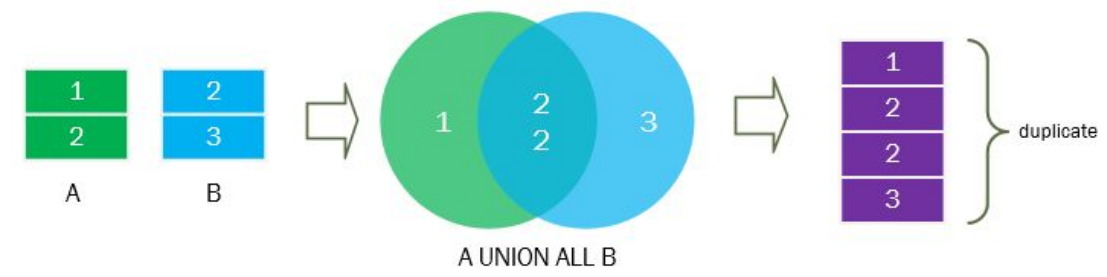
UNION & UNION ALL

Both UNION and UNION ALL concatenate the result of two different SQLs

- UNION: eliminate any duplicate rows
- UNION ALL: does not remove duplicates

```
SELECT column_list
FROM table1
UNION ALL
SELECT column_list
FROM table2
```





UNION & UNION ALL

- Use to combine data from multiple tables with same structure
- For example:
  - Table 1: campaign_id, order_id, ads_cost, grand_total (Facebook)
  - Table 1: campaign_id, order_id, ads_cost, grand_total (Google)

```
SELECT order_id, ads_cost, 'google' source
FROM google
UNION ALL
SELECT order_id, ads_cost, 'facebook' source
FROM facebook
```



200Lab Education

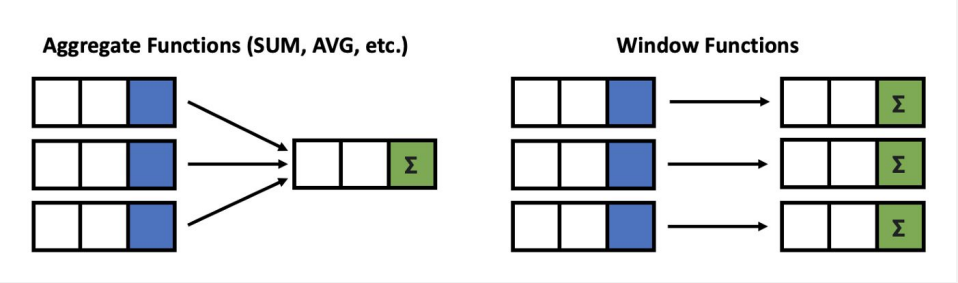
Nga Vu - Data Mentor

Analytic Functions

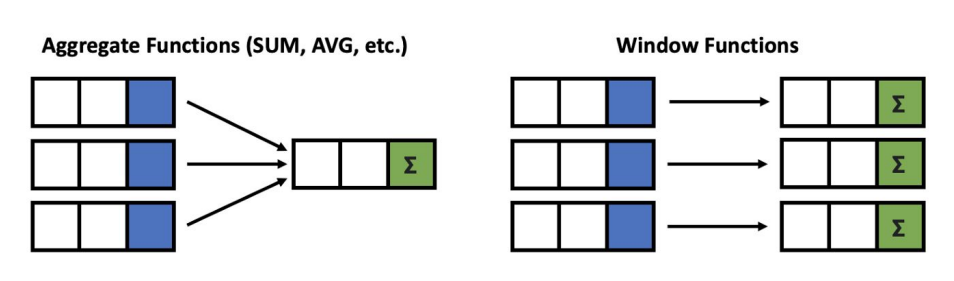
Window Function

Analytic Function Concept

- Like **aggregate functions**, **analytic functions** return aggregate results, but analytics do not group the result set.
- Instead, they return the group value multiple times with each record, allowing further analysis.
- **Window Function = Analytic Function**



Analytic Function Concept



year	country	product	profit
2000	Finland	Computer	1500
2000	Finland	Phone	100
2001	Finland	Phone	10
2000	India	Calculator	75
2000	India	Calculator	75
2000	India	Computer	1200
2000	USA	Calculator	75
2000	USA	Computer	1500
2001	USA	Calculator	50
2001	USA	Computer	1500
2001	USA	Computer	1200
2001	USA	TV	150
2001	USA	TV	100

country	country_profit
Finland	1610
India	1350
USA	4575

year	country	product	profit	total_profit	country_profit
2000	Finland	Computer	1500	7535	1610
2000	Finland	Phone	100	7535	1610
2001	Finland	Phone	10	7535	1610
2000	India	Calculator	75	7535	1350
2000	India	Calculator	75	7535	1350
2000	India	Computer	1200	7535	1350
2000	USA	Calculator	75	7535	4575
2000	USA	Computer	1500	7535	4575
2001	USA	Calculator	50	7535	4575
2001	USA	Computer	1200	7535	4575
2001	USA	Computer	1500	7535	4575
2001	USA	TV	100	7535	4575
2001	USA	TV	150	7535	4575

Analytic Functions - Syntax

```
SELECT column_list,
       window_function_name(expression)
OVER (
  PARTITION BY colum_list,
  ORDER BY colum_list)
FROM table_name
```

```
SELECT   year, country, product, profit,
       SUM(profit) OVER() AS total_profit,
       SUM(profit) OVER(PARTITION BY
country) AS country_profit
     FROM sales
     ORDER BY country, year, product, profit;
```

year	country	product	profit	total_profit	country_profit
2000	Finland	Computer	1500	7535	1610
2000	Finland	Phone	100	7535	1610
2001	Finland	Phone	10	7535	1610
2000	India	Calculator	75	7535	1350
2000	India	Calculator	75	7535	1350
2000	India	Computer	1200	7535	1350
2000	USA	Calculator	75	7535	4575
2000	USA	Computer	1500	7535	4575
2001	USA	Calculator	50	7535	4575
2001	USA	Computer	1200	7535	4575
2001	USA	Computer	1500	7535	4575
2001	USA	TV	100	7535	4575
2001	USA	TV	150	7535	4575

Analytic Functions

Name	Description
CUME_DIST()	Cumulative distribution value
DENSE_RANK()	Rank of current row within its partition, without gaps
FIRST_VALUE()	Value of argument from first row of window frame
LAG()	Value of argument from row lagging current row within partition
LAST_VALUE()	Value of argument from last row of window frame
LEAD()	Value of argument from row leading current row within partition
NTH_VALUE()	Value of argument from N-th row of window frame
NTILE()	Bucket number of current row within its partition.
PERCENT_RANK()	Percentage rank value
RANK()	Rank of current row within its partition, with gaps
ROW_NUMBER()	Number of current row within its partition

Analytic Function + Aggregate Function

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        e.officeCode ,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2)
SELECT *, SUM(total) OVER(PARTITION BY officeCode) total_revenue,
total/(SUM(total) OVER(PARTITION BY officeCode)) contribution
FROM cte_emp_sum
ORDER BY officeCode
```

	employeeNumber	employeeName	total
4	1,504	Barry Jones	704,853.91
5	1,165	Leslie Jennings	1,081,530.54
6	1,323	George Vanauf	669,377.05
7	1,401	Pamela Castillo	868,220.55
8	1,337	Loui Bondur	569,485.75
9	1,621	Mami Nishi	457,110.07
10	1,286	Foon Yue Tseng	488,212.67



	employeeNumber	employeeName	total	total_revenue	contribution
1	1,370	Gerard Hernandez	1,258,577.81	9,604,190.61	0.131045
2	1,165	Leslie Jennings	1,081,530.54	9,604,190.61	0.11261
3	1,401	Pamela Castillo	868,220.55	9,604,190.61	0.0904
4	1,501	Larry Bott	732,096.79	9,604,190.61	0.076227
5	1,504	Barry Jones	704,853.91	9,604,190.61	0.07339
6	1,323	George Vanauf	669,377.05	9,604,190.61	0.069696

Analytic Function + Aggregate Function

200Lab Education

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2)
SELECT *, SUM(total) OVER() total_revenue,
total/(SUM(total) OVER()) contribution
FROM cte_emp_sum
```

	employeeNumber	employeeName	officeCode	total
1	1,370	Gerard Hernandez	4	1,258,577.81
2	1,166	Leslie Thompson	1	347,533.03
3	1,611	Andy Fixter	6	562,582.59
4	1,504	Barry Jones	7	704,853.91



employeeName	officeCode	total	total_revenue	contribution
Barry Jones	7	704,853.91	1,436,950.7	0.490521
Larry Bott	7	732,096.79	1,436,950.7	0.509479
Andy Fixter	6	562,582.59	1,147,176.35	0.490406
Peter Marsh	6	584,593.76	1,147,176.35	0.509594
Mami Nishi	5	457,110.07	457,110.07	1
Gerard Hernandez	4	1,258,577.81	3,083,761.58	0.408131
Pamela Castillo	4	868,220.55	3,083,761.58	0.281546

Analytic Function: ROW_NUMBER

Returns the number of the current row within its partition. Rows numbers range from 1 to the number of partition rows.

ORDER BY affects the order in which rows are numbered

	employeeNumber	employeeName	total
4	1,504	Barry Jones	704,853.91
5	1,165	Leslie Jennings	1,081,530.54
6	1,323	George Vanauf	669,377.05
7	1,401	Pamela Castillo	868,220.55
8	1,337	Loui Bondur	569,485.75
9	1,621	Mami Nishi	457,110.07
10	1,286	Foon Yue Tseng	488,212.67



	employeeNumber	employeeName	total	row_num
1	1,370	Gerard Hernandez	1,258,577.81	1
2	1,165	Leslie Jennings	1,081,530.54	2
3	1,401	Pamela Castillo	868,220.55	3
4	1,501	Larry Bott	732,096.79	4
5	1,504	Barry Jones	704,853.91	5
6	1,323	George Vanauf	669,377.05	6

Analytic Function: ROW_NUMBER

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2)
SELECT *, ROW_NUMBER() OVER(ORDER BY total DESC) row_num
FROM cte_emp_sum
```

	employeeNumber	employeeName	total
4	1,504	Barry Jones	704,853.91
5	1,165	Leslie Jennings	1,081,530.54
6	1,323	George Vanauf	669,377.05
7	1,401	Pamela Castillo	868,220.55
8	1,337	Loui Bondur	569,485.75
9	1,621	Mami Nishi	457,110.07
10	1,286	Foon Yue Tseng	488,212.67



	employeeNumber	employeeName	total	row_num
1	1,370	Gerard Hernandez	1,258,577.81	1
2	1,165	Leslie Jennings	1,081,530.54	2
3	1,401	Pamela Castillo	868,220.55	3
4	1,501	Larry Bott	732,096.79	4
5	1,504	Barry Jones	704,853.91	5
6	1,323	George Vanauf	669,377.05	6

Analytic Function: ROW_NUMBER

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        YEAR(o.orderDate) year,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2,3),
cte_emp_rn AS
(SELECT *, ROW_NUMBER() OVER( PARTITION BY year
           ORDER BY total DESC) row_num
FROM cte_emp_sum)
SELECT * FROM cte_emp_rn
WHERE row_num <=5
```

	employeeNumber	employeeName	total
4	1,504	Barry Jones	704,853.91
5	1,165	Leslie Jennings	1,081,530.54
6	1,323	George Vanauf	669,377.05
7	1,401	Pamela Castillo	868,220.55
8	1,337	Loui Bondur	569,485.75
9	1,621	Mami Nishi	457,110.07
10	1,286	Foon Yue Tseng	488,212.67

	employeeNumber	employeeName	year	total	row_num
1	1,165	Leslie Jennings	2,003	413,219.85	1
2	1,401	Pamela Castillo	2,003	317,104.78	2
3	1,370	Gerard Hernandez	2,003	295,246.44	3
4	1,504	Barry Jones	2,003	288,014.99	4
5	1,621	Mami Nishi	2,003	267,249.4	5
6	1,370	Gerard Hernandez	2,004	534,575.67	1
7	1,323	George Vanauf	2,004	428,062.73	2
8	1,401	Pamela Castillo	2,004	409,910.07	3
9	1,504	Barry Jones	2,004	388,872.38	4
10	1,612	Peter Marsh	2,004	338,782.84	5

Analytic Function: RANK vs DENSE_RANK

RANK: Returns the rank of the current row within its partition, with gaps
DENSE_RANK: Returns the rank of the current row within its partition, without gaps

ORDER BY affects the order in which rows are ranked

The **row_number** gives continuous numbers, while **rank** and **dense_rank** give the same rank for duplicates

```
mysql> SELECT
         val,
         ROW_NUMBER() OVER w AS 'row_number',
         RANK()       OVER w AS 'rank',
         DENSE_RANK() OVER w AS 'dense_rank'
       FROM numbers
       WINDOW w AS (ORDER BY val);
+------+------------+------+------------+
| val  | row_number | rank | dense_rank |
+------+------------+------+------------+
|    1 |          1 |    1 |          1 |
|    1 |          2 |    1 |          1 |
|    2 |          3 |    3 |          2 |
|    3 |          4 |    4 |          3 |
|    3 |          5 |    4 |          3 |
|    3 |          6 |    4 |          3 |
|    4 |          7 |    7 |          4 |
|    4 |          8 |    7 |          4 |
|    5 |          9 |    9 |          5 |
+------+------------+------+------------+
```

Analytic Function: RANK vs DENSE_RANK

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        YEAR(o.orderDate) year,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2,3),
cte_emp_rn AS
(SELECT *, RANK() OVER( PARTITION BY year
                ORDER BY total DESC) rank_num
FROM cte_emp_sum)
SELECT * FROM cte_emp_rn
WHERE row_num <=5
```

	employeeNumber	employeeName	total
4	1,504	Barry Jones	704,853.91
5	1,165	Leslie Jennings	1,081,530.54
6	1,323	George Vanauf	669,377.05
7	1,401	Pamela Castillo	868,220.55
8	1,337	Loui Bondur	569,485.75
9	1,621	Mami Nishi	457,110.07
10	1,286	Foon Yue Tseng	488,212.67

	employeeNumber	employeeName	year	total	row_num
1	1,165	Leslie Jennings	2,003	413,219.85	1
2	1,401	Pamela Castillo	2,003	317,104.78	2
3	1,370	Gerard Hernandez	2,003	295,246.44	3
4	1,504	Barry Jones	2,003	288,014.99	4
5	1,621	Mami Nishi	2,003	267,249.4	5
6	1,370	Gerard Hernandez	2,004	534,575.67	1
7	1,323	George Vanauf	2,004	428,062.73	2
8	1,401	Pamela Castillo	2,004	409,910.07	3
9	1,504	Barry Jones	2,004	388,872.38	4
10	1,612	Peter Marsh	2,004	338,782.84	5

Analytic Function: LAG vs LEAD

LAG: Returns the value of expr from the row that lags (precedes) the current row by N rows within its partition.

LEAD: Returns the value of expr from the row that leads (follows) the current row by N rows within its partition

ORDER BY affects the row order

```
mysql> SELECT
         t, val,
         LAG(val)        OVER w AS 'lag',
         LEAD(val)       OVER w AS 'lead',
         val - LAG(val)  OVER w AS 'lag diff',
         val - LEAD(val) OVER w AS 'lead diff'
       FROM series
       WINDOW w AS (ORDER BY t);
+----------+-----+------+------+----------+-----------+
| t        | val | lag  | lead | lag diff | lead diff |
+----------+-----+------+------+----------+-----------+
| 12:00:00 | 100 | NULL | 125  |     NULL |       -25 |
| 13:00:00 | 125 | 100  | 132  |       25 |        -7 |
| 14:00:00 | 132 | 125  | 145  |        7 |       -13 |
| 15:00:00 | 145 | 132  | 140  |       13 |         5 |
| 16:00:00 | 140 | 145  | 150  |       -5 |       -10 |
| 17:00:00 | 150 | 140  | 200  |       10 |       -50 |
| 18:00:00 | 200 | 150  | NULL |       50 |      NULL |
+----------+-----+------+------+----------+-----------+
```

Analytic Function: LAG vs LEAD

```
WITH cte_prodline_sum AS
(SELECT p.productLine,
        YEAR(o.orderDate) order_year,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM orders o
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
      JOIN products p ON p.productCode =od.productCode
 GROUP BY 1,2),
cte_prodline_sum2 AS
(SELECT *,
LAG(total) OVER (PARTITION BY productLine ORDER BY order_year) last_year_total
FROM cte_prodline_sum
ORDER BY productLine,order_year)

SELECT *,
       (total-last_year_total) growth,
       (total-last_year_total)/last_year_total growth_rate
FROM cte_prodline_sum2
```

	productLine	order_year	total	last_year_total	growth	growth_rate
12	Ships	2,005	104,490.16	337,326.1	-232,835.94	-0.69024
13	Trains	2,003	65,822.05	[NULL]	[NULL]	[NULL]
14	Trains	2,004	96,285.53	65,822.05	30,463.48	0.462816
15	Trains	2,005	26,425.34	96,285.53	-69,860.19	-0.725552
16	Trucks and Buses	2,003	376,657.12	[NULL]	[NULL]	[NULL]
17	Trucks and Buses	2,004	465,390	376,657.12	88,732.88	0.23558
18	Trucks and Buses	2,005	182,066.45	465,390	-283,323.55	-0.608787
19	Vintage Cars	2,003	619,161.48	[NULL]	[NULL]	[NULL]
20	Vintage Cars	2,004	854,551.85	619,161.48	235,390.37	0.380176
21	Vintage Cars	2,005	323,846.3	854,551.85	-530,705.55	-0.621034

Analytic Function: USE LEFT JOIN INSTEAD

```
WITH cte_prodline_sum AS
(SELECT p.productLine,
        YEAR(o.orderDate) order_year,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM orders o
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
      JOIN products p ON p.productCode =od.productCode
 GROUP BY 1,2)

 SELECT ps1.*,ps2.total last_year_total
FROM cte_prodline_sum ps1 LEFT JOIN cte_prodline_sum ps2
      ON ps1.productLine = ps2.productLine and ps1.order_year-1 = ps2.order_year
ORDER BY 1,2

SELECT *,
      (total-last_year_total) growth,
      (total-last_year_total)/last_year_total growth_rate
FROM cte_prodline_sum2
```

	productLine	order_year	total	last_year_total	growth	growth_rate
12	Ships	2,005	104,490.16	337,326.1	-232,835.94	-0.69024
13	Trains	2,003	65,822.05	[NULL]	[NULL]	[NULL]
14	Trains	2,004	96,285.53	65,822.05	30,463.48	0.462816
15	Trains	2,005	26,425.34	96,285.53	-69,860.19	-0.725552
16	Trucks and Buses	2,003	376,657.12	[NULL]	[NULL]	[NULL]
17	Trucks and Buses	2,004	465,390	376,657.12	88,732.88	0.23558
18	Trucks and Buses	2,005	182,066.45	465,390	-283,323.55	-0.608787
19	Vintage Cars	2,003	619,161.48	[NULL]	[NULL]	[NULL]
20	Vintage Cars	2,004	854,551.85	619,161.48	235,390.37	0.380176
21	Vintage Cars	2,005	323,846.3	854,551.85	-530,705.55	-0.621034

Analytic Function: Window Function Frame

A frame is a **subset of the current partition** and the frame clause specifies how to define the subs

By defining a frame to be all rows from the partition start to the current row, you can **compute running totals** for each row.

By defining a frame as extending N rows on eithe side of the current row, you can **compute rolling averages**

period	shop	revenue	rows_cumulative_revenue
2021/04	Shop 1	154,552.82	
2021/05	Shop 1	1,489,774.16	
2021/06	Shop 1	1,845,662.35	
2021/06	Shop 1	52,489.35	
2021/04	Shop 2	341,227.53	
2021/04	Shop 2	21,487.63	
2021/05	Shop 2	315,447.24	
2021/05	Shop 2	6,548.49	
2021/06	Shop 2	387,779.49	

Analytic Function: Window Function Frame

Syntax

```
frame_clause:
  {ROWS | RANGE} {frame_border |
BETWEEN frame_border AND
frame_border}

frame_border:
  | UNBOUNDED PRECEDING
  | UNBOUNDED FOLLOWING
  | CURRENT ROW
  | expr PRECEDING
  | expr FOLLOWING
```

ROWS: The frame is defined by beginning and ending row positions.

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)

RANGE: The frame is defined by rows within a value range.

RANGE BETWEEN 6 PRECEDING AND CURRENT ROW

CURRENT ROW: For ROWS, the bound is the current row. For RANGE, the bound is the peers of the current row.

UNBOUNDED PRECEDING: The bound is the first partition row.

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

UNBOUNDED FOLLOWING: The bound is the last partition row.

RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

Analytic Function: Window Function Frame

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        EXTRACT( YEAR_MONTH FROM o.orderDate) date,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2,3)

SELECT *, SUM(total) OVER(PARTITION BY employeeNumber ORDER BY
date
                RANGE BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) running_total
FROM cte_emp_sum
```

	employeeNumber	employeeName	date	total	running_total
1	1,165	Leslie Jennings	200,303	27,582.15	27,582.15
2	1,165	Leslie Jennings	200,307	94,276.97	121,859.12
3	1,165	Leslie Jennings	200,308	56,052.56	177,911.68
4	1,165	Leslie Jennings	200,309	29,997.09	207,908.77
5	1,165	Leslie Jennings	200,310	136,302.77	344,211.54
6	1,165	Leslie Jennings	200,311	45,084.38	389,295.92
7	1,165	Leslie Jennings	200,312	23,923.93	413,219.85
8	1,165	Leslie Jennings	200,402	12,573.28	425,793.13

Analytic Function: ROWS vs RANGE

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        o.orderDate date,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2,3)

SELECT *, AVG(total) OVER(PARTITION BY employeeNumber ORDER BY
date
                RANGE BETWEEN INTERVAL '3' DAY PRECEDING AND
                INTERVAL '1' DAY PRECEDING) moving_avg_3day
FROM cte_emp_sum
```

	employeeNumber	employeeName	date	total	moving_avg_3day
1	1,165	Leslie Jennings	2003-03-25	16,537.85	[NULL]
2	1,165	Leslie Jennings	2003-03-26	11,044.3	16,537.85
3	1,165	Leslie Jennings	2003-07-02	55,601.84	[NULL]
4	1,165	Leslie Jennings	2003-07-24	38,675.13	[NULL]
5	1,165	Leslie Jennings	2003-08-08	56,052.56	[NULL]
6	1,165	Leslie Jennings	2003-09-12	29,997.09	[NULL]
7	1,165	Leslie Jennings	2003-10-10	54,682.68	[NULL]

Analytic Function: ROW vs RANGE

```
WITH cte_emp_sum AS
(SELECT e.employeeNumber,
        CONCAT(e.firstName,' ',e.lastName) employeeName ,
        o.orderDate date,
        SUM(od.priceEach*od.quantityOrdered) total
 FROM employees e JOIN customers c
      ON e.employeeNumber = c.salesRepEmployeeNumber
      JOIN orders o ON o.customerNumber = c.customerNumber
      JOIN orderdetails od ON od.orderNumber = o.orderNumber
 GROUP BY 1,2,3)

SELECT *, AVG(total) OVER(PARTITION BY employeeNumber ORDER BY
date
                ROWS BETWEEN 3 PRECEDING AND  1 PRECEDING)
moving_avg_3day
FROM cte_emp_sum
```

	employeeNumber	employeeName	date	total	moving_avg_3day
1	1,165	Leslie Jennings	2003-03-25	16,537.85	[NULL]
2	1,165	Leslie Jennings	2003-03-26	11,044.3	16,537.85
3	1,165	Leslie Jennings	2003-07-02	55,601.84	13,791.075
4	1,165	Leslie Jennings	2003-07-24	38,675.13	27,727.996667
5	1,165	Leslie Jennings	2003-08-08	56,052.56	35,107.09
6	1,165	Leslie Jennings	2003-09-12	29,997.09	50,109.843333
7	1,165	Leslie Jennings	2003-10-10	54,682.68	41,574.926667

Analytic Function: RANGE & PARTITION Function

Partition

- `CUME_DIST()`
- `DENSE_RANK()`
- `LAG()`
- `LEAD()`
- `NTILE()`
- `PERCENT_RANK()`
- `RANK()`
- `ROW_NUMBER()`

Frame

- `FIRST_VALUE()`
- `LAST_VALUE()`
- `NTH_VALUE()`